

Conversation QA with RAG Chatbot

```
!pip install langchain-google-genai
!pip install sentence-transformers langchain-huggingface
!pip install langchain-community
!pip install langchain-Chroma
!pip install pypdf
```

[Show hidden output](#)

```
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_community.document_loaders import WebBaseLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFLoader
from langchain_chroma import Chroma
import bs4
```

```
from dotenv import load_dotenv
load_dotenv()
from google.colab import userdata
import os
```

```
os.environ['GOOGLE_APPLICATION_CREDENTIALS'] = userdata.get('GEMINI')
os.environ['HUGGINGFACEHUB_API_TOKEN'] = userdata.get('HB_TOKEN')
```

with_structure_output

```
from pydantic import BaseModel, Field

class RagResponse(BaseModel):
    answer:str=Field(description="Answer the user question based only on the retrieved context")
    confidence:float=Field(description="Confidence in the answer from 0 to 1")
```

with structured output and generation evalauation

```
"""
from pydantic import BaseModel, Field

GenerationEvaluation
class RagResponse(BaseModel):
    answer:str=Field(description="Answer the user question based only on the retrieved context")
    confidence:float=Field(description="Confidence in the answer from 0 to 1")

    faithfulness: float = Field(
        description="How well the answer is supported by the retrieved context, 0 to 1"
    )

    answer_relevance: float = Field(
        description="How directly the answer addresses the question, 0 to 1"
    )

    correctness: float = Field(
        description="How closely the answer matches the ground truth, 0 to 1"
    )

    explanation: str = Field(
        description="Brief explanation of the scores"
    )
"""
```

```
'\nfrom pydantic import BaseModel, Field\n\nGenerationEvaluation\n\nclass RagResponse(BaseModel):\n    answer:str=Field(description="Answer the user question based only on the retrieved context")\n    confidence:float=Field(description="Confidence in the answer from 0 to 1")\n\n    faithfulness: float = Field(\n        description="How well the answer is supported by the retrieved context, 0 to 1"\n    )\n\n    answer_relevance: float = Field(\n        description="How directly the answer addresses the question, 0 to 1"\n    )\n\n    correctness: float = Field(\n        description="How closely the answer matches the ground truth,
```

```
from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI( model="gemini-3.6-flash",
                             api_key=userdata.get('GEMINI'),
                             max_tokens=500,
```

```

        timeout = None,
        max_retries = 3,
        tempreture=0)
llm = llm.with_structured_output(RagResponse)

```

WARNING:langchain_google_genai.chat_models:Unexpected argument 'tempreture' provided to ChatGoogleGenerativeAI. Did you mean: 'temperature'?
 /usr/local/lib/python3.13/dist-packages/IPython/core/interactiveshell.py:3553: UserWarning: WARNING! tempreture is not default parameter
 tempreture was transferred to model_kwargs.
 Please confirm that tempreture is what you intended.
 exec(code_obj, self.user_global_ns, self.user_ns)

```

from langchain_huggingface import HuggingFaceEmbeddings
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

```

Loading weights: 100%

103/103 [00:00<00:00, 1545.27it/s]

Load, Chunk and index document

```

#loader = WebBaseLoader(web_paths="https://docs.langchain.com/"),
#bs_kwargs=dict(
#    #parse_only=bs4.SoupStrainer(
#        #class_=("post-content", "post-title", "post-header")))

```

```

loader = PyPDFLoader("/content/ARBL Annual-Report-2013-14.pdf")
docs = loader.load()
text_splitter = RecursiveCharacterTextSplitter(chunk_size=400, chunk_overlap=50,length_function=len,strip_whitespace=True)
split = text_splitter.split_documents(docs)
vectorstore = Chroma.from_documents(documents=split, embedding=embeddings)
retriever = vectorstore.as_retriever(search_type="similarity",search_kwargs={"k":5})
retriever

```

VectorStoreRetriever(tags=['Chroma', 'HuggingFaceEmbeddings'], vectorstore=<langchain_chroma.vectorstores.Chroma object at 0x7eada7864910>, search_kwargs={'k': 5})

Retriever evalaution

```
!pip install ragas
```

[Show hidden output](#)

Eval Dataset

```

eval_data = [
    {
        "question": "What has worked for Amara Raja in the recent past will continue to work over the foreseeable future with",
        "relevant_page": 10
    },
    {
        "question": "We continue to build our capability matrix for we truly believe that there is always a 'Gotta be a better",
        "relevant_page": 11
    }
]

```

```
retriever = vectorstore.as_retriever(search_type="similarity",search_kwargs={"k":2})
```

```

for item in eval_data:
    docs = retriever.invoke(item["question"])
    print("\nQuestion:",item["question"])

    for doc in docs:
        print("\nPage:",doc.metadata.get("page"))
        print("Content:",doc.page_content[:200])

```

Question: "What has worked for Amara Raja in the recent past will continue to work over the foreseeable future with one differen

Page: 10
 Content: 2001-2011. India's preference
 "What has worked for Amara Raja in the recent past will
 continue to work over the foreseeable future with one

difference - the scale and urgency will increase, transla

Page: 10

Content: 2001-2011. India's preference

"What has worked for Amara Raja in the recent past will continue to work over the foreseeable future with one difference - the scale and urgency will increase, transla

Question: "We continue to build our capability matrix for we truly believe that there is always a 'Gotta be a better way' in del

Page: 11

Content: investments with de-risking strategies.

The investment has been financed entirely from accruals and no debt through which we hope to retain business profitability.

THE MANAGEMENT STATEMENT

"We con

Page: 11

Content: investments with de-risking strategies.

The investment has been financed entirely from accruals and no debt through which we hope to retain business profitability.

THE MANAGEMENT STATEMENT

"We con

```
def recal_at_k(retrieved_docs, relevant_page, k):
    retrived_pages = [
        doc.metadata.get("page")
        for doc in retrieved_docs[:k]
    ]
    return int(relevant_page in retrived_pages)
```

```
scores = []

for item in eval_data:
    docs = retriever.invoke(item["question"])
    scores.append(recal_at_k(docs, item["relevant_page"], 2))
print("Recall@2:", sum(scores)/len(scores))
```

Recall@2: 1.0

Precision@k: Of the documents i retrived, how many are relevant

```
def precision_at_k(retrieved_docs, relevant_pages, k):
    retrived_pages = [
        doc.metadata.get("page")
        for doc in retrieved_docs[:k]
    ]
    relevant_count = sum(page in relevant_pages for page in retrived_pages)

    return relevant_count/k
```

```
score = precision_at_k(docs, relevant_pages=[11], k=2)
print(score)
```

1.0

MRR: Mean Reciprokal Rank tells how high the first relevant document appears

```
def reciprocal_rank(retrieved_docs, relevant_pages):

    for rank, doc in enumerate(retrieved_docs, start=1):

        page = doc.metadata.get("page")

        if page in relevant_pages:
            return 1 / rank

    return 0
```

```

scores = []

for item in eval_data:

    docs = retriever.invoke(item["question"])

    score = reciprocal_rank(
        docs,
        [item["relevant_page"]] # Corrected key and wrapped in a list
    )

    scores.append(score)

mrr = sum(scores) / len(scores)

print("MRR:", mrr)

```

MRR: 1.0

Evaluate Retriever

```

def evaluate_retriever(retriever, eval_data, k=3):

    recall_scores = []
    precision_scores = []
    reciprocal_ranks = []

    for item in eval_data:

        docs = retriever.invoke(item["question"])

        relevant_pages = item["relevant_pages"]

        retrieved_pages = [
            doc.metadata.get("page")
            for doc in docs[:k]
        ]

        # Recall@K
        recall = int(
            any(page in relevant_pages for page in retrieved_pages)
        )

        # Precision@K
        relevant_count = sum(
            page in relevant_pages
            for page in retrieved_pages
        )

        precision = relevant_count / k

        # MRR
        rr = 0

        for rank, page in enumerate(retrieved_pages, start=1):

            if page in relevant_pages:
                rr = 1 / rank
                break

        recall_scores.append(recall)
        precision_scores.append(precision)
        reciprocal_ranks.append(rr)

    return {
        "Recall@K": sum(recall_scores) / len(recall_scores),
        "Precision@K": sum(precision_scores) / len(precision_scores),
        "MRR": sum(reciprocal_ranks) / len(reciprocal_ranks)
    }

```

```

def evaluate_retriever(retriever, eval_data, k=3):

    recall_scores = []
    precision_scores = []
    reciprocal_ranks = []

```

```

for item in eval_data:

    docs = retriever.invoke(item["question"])

    # Corrected: Access 'relevant_page' and wrap it in a list
    relevant_pages = [item["relevant_page"]]

    retrieved_pages = [
        doc.metadata.get("page")
        for doc in docs[:k]
    ]

    # Recall@K
    recall = int(
        any(page in relevant_pages for page in retrieved_pages)
    )

    # Precision@K
    relevant_count = sum(
        page in relevant_pages
        for page in retrieved_pages
    )

    precision = relevant_count / k

    # MRR
    rr = 0

    for rank, page in enumerate(retrieved_pages, start=1):

        if page in relevant_pages:
            rr = 1 / rank
            break

    recall_scores.append(recall)
    precision_scores.append(precision)
    reciprocal_ranks.append(rr)

    return {
        "Recall@K": sum(recall_scores) / len(recall_scores),
        "Precision@K": sum(precision_scores) / len(precision_scores),
        "MRR": sum(reciprocal_ranks) / len(reciprocal_ranks)
    }

results = evaluate_retriever(
    retriever,
    eval_data,
    k=3
)

print(results)

{'Recall@K': 1.0, 'Precision@K': 0.6666666666666666, 'MRR': 1.0}

```

Add Chat History: History Aware Retriever

```

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.prompts import MessagesPlaceholder

```

```
!pip install langchain-classic
```

[Show hidden output](#)

```
from langchain_classic.chains import create_history_aware_retriever
```

```

contextualize_q_system_prompt = """ Given chat history and latest user question which might reference context in the chat history,
formulate standalone question which can be understood without the chat history. Do NOT answer the question,
just formulate it if needed otherwise return it as it is. """

contextualize_q_prompt = ChatPromptTemplate.from_messages([
    ("system", contextualize_q_system_prompt),
    MessagesPlaceholder("chat_history"),
    ("human", "{input}"),
])

```

```

    ])
    history_aware_retriever = create_history_aware_retriever(
        llm,
        retriever,
        contextualize_q_prompt
    )

```

QA Prompt with chat history

```

from langchain_classic.chains.combine_documents import create_stuff_documents_chain
from langchain_classic.chains import create_retrieval_chain

```

```

# Prompt template
system_prompt = """you are an assistance for question-answering task. use the following pieces of retrieved context to answer the
                    If you don't know the answer say that you don't know, use the three sentences maximum and keep the answer concise

                    {context}"""
prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("human", "{question}"),
])

```

```

qa_prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    MessagesPlaceholder("chat_history"),
    ("human", "{input}"),
])

question_answer_chain = create_stuff_documents_chain(llm, qa_prompt)
rag_chain = create_retrieval_chain(history_aware_retriever, question_answer_chain)

```

manage chat history automatically

```

from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_core.chat_history import BaseChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

```

```

store = {}

def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = ChatMessageHistory()
    return store[session_id]

conversation_rag_chain = RunnableWithMessageHistory(
    rag_chain,
    get_session_history,
    input_messages_key="input",
    history_messages_key="chat_history",
    output_messages_key="answer"
)

```

```

/usr/local/lib/python3.13/dist-packages/IPython/core/interactiveshell.py:3553: LangChainDeprecationWarning: RunnableWithMessageHistory
exec(code_obj, self.user_global_ns, self.user_ns)

```

Generation Evaluation

```

eval_data = [
    {
        "question": "Amara Raja has not tapped the equity market even once during its high-growth phase that commenced in FY01.",
        "ground_truth": "Market capitalisation grew at 84.75% CAGR over five years enhancing shareholders wealth."
    },
    {
        "question": "India became the world's third biggest economy in terms of",
        "ground_truth": "Purchasing power parity (PPP), according to a World Bank report, rising from the tenth position in 2000."
    }
]

```

```

results = []

```

```
for item in eval_data:
    response = conversation_rag_chain.invoke(
        {"input":item["question"]},
        config={"configurable":{"session_id":"user_001"}}
    )

    results.append({"question:" == item["question"],
                    "answer:" == response["answer"],
                    "ground_truth:" == item["ground_truth"]
                  })

print("Accuracy:",sum(results)/len(results))
```

Example invoke

```
conversation_rag_chain.invoke(
    {"input":"What was my last question?"},
    config={"configurable":{"session_id":"user_002"}}
)["answer"]
```

It's able to remember the history and context and passed retrieval and generation eval